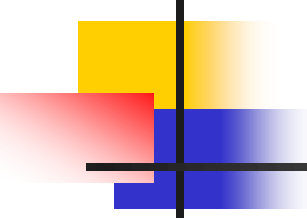


ECSE 4320/6320 Advanced Computer Systems

Thread 1: How to Speed Up Computing? Part 3



Interconnect Within Server

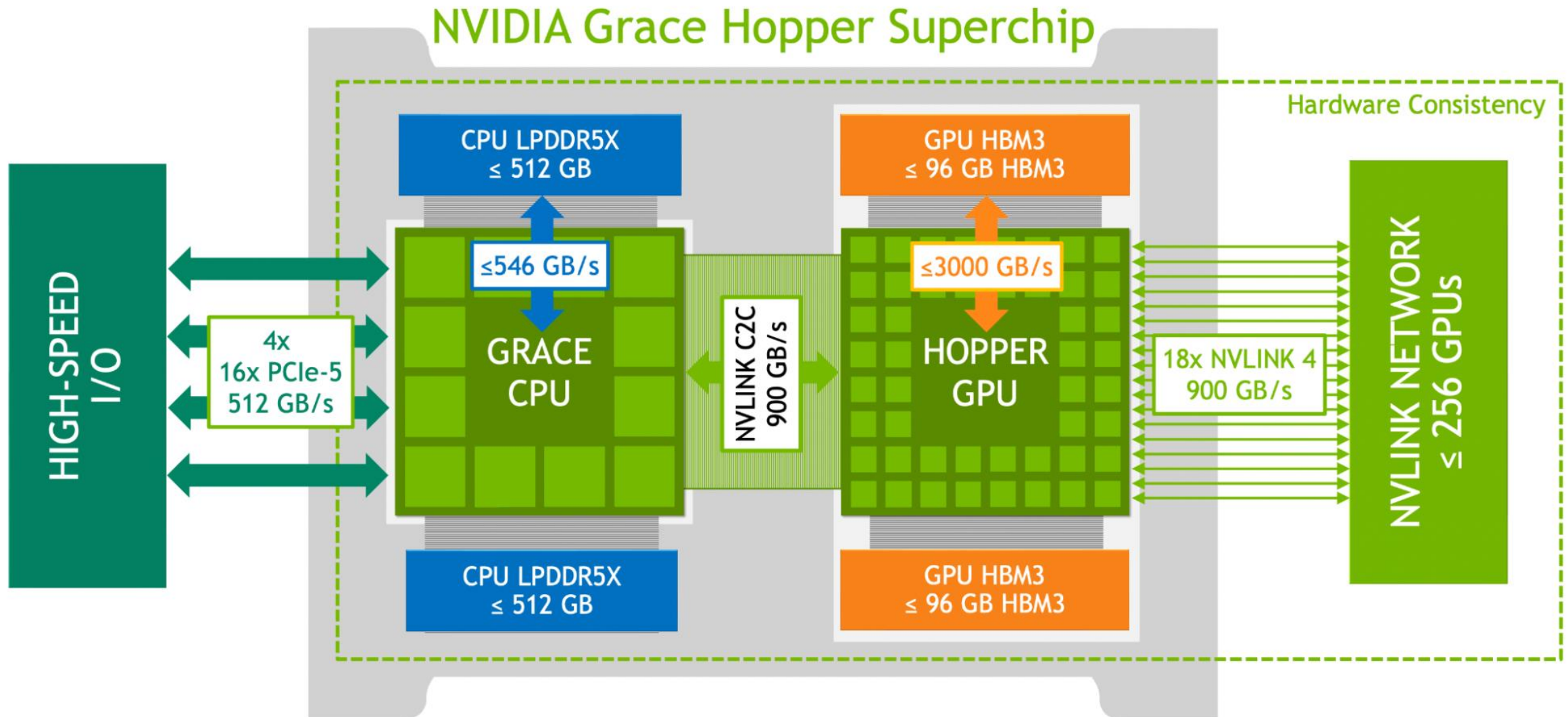
❑ Parallel link: DDR

- Wide, source-synchronous parallel bus from CPU memory controller to DRAM DIMMs
- Fixed 64-bit per channel (plus ECC), very short reach on the motherboard
- Lowest latency (ns), tens of GB/s per channel (DDR5 → DDR6 → ...)

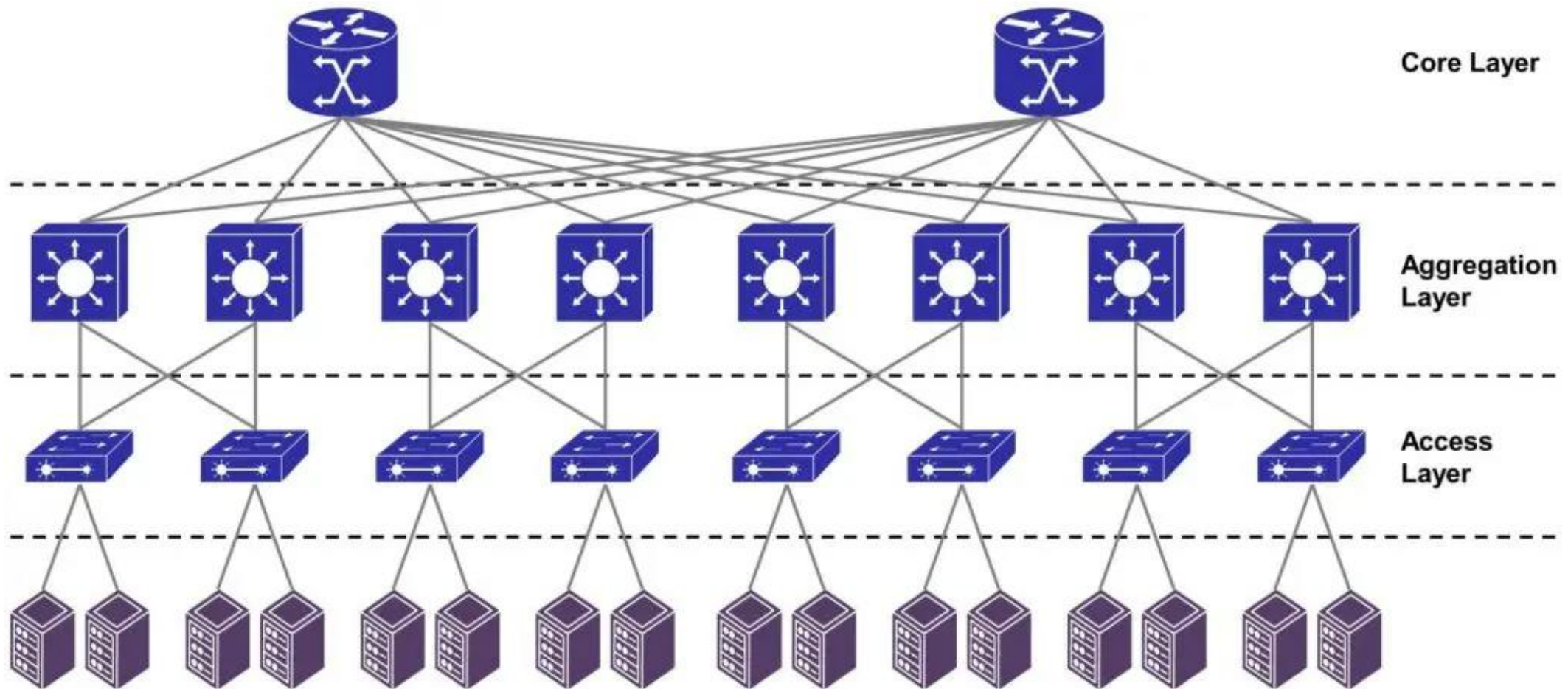
❑ Serial link: PCIe, NVLink

- Sends data as a high-speed bit stream over a few differential pairs using SerDes and clock-data recovery (CDR); lanes can be bonded (x1...x16) for more bandwidth
- Uses framing/encoding (e.g., 128b/130b, PAM4 on newer gens), link training and equalization for signal integrity; includes flow control and CRC/retry
- PCIe: universal CPU-to-device I/O bus that connects GPUs, SSDs, and NICs
- NVLink: NVIDIA's link technology for direct GPU↔GPU (and CPU↔GPU) link within a node.

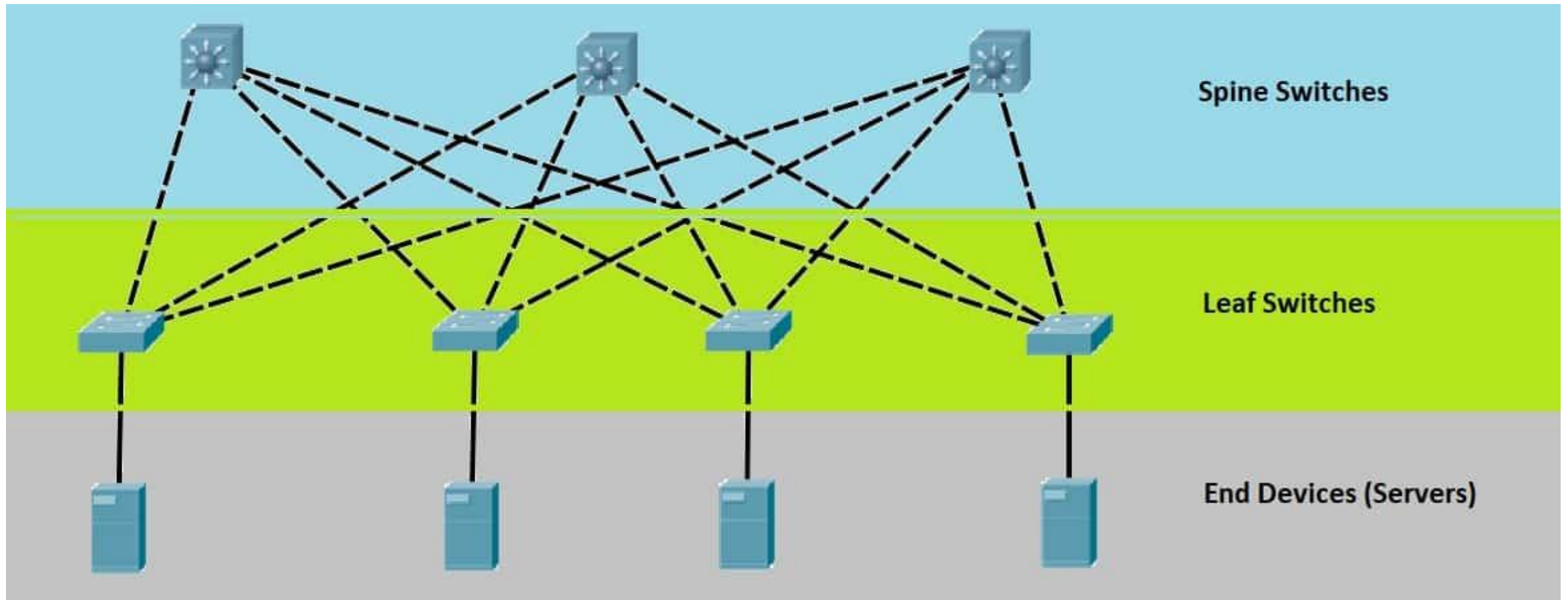
Collaboration Among Multiple CPUs/GPUs

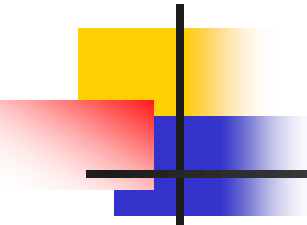


Interconnect Beyond Server



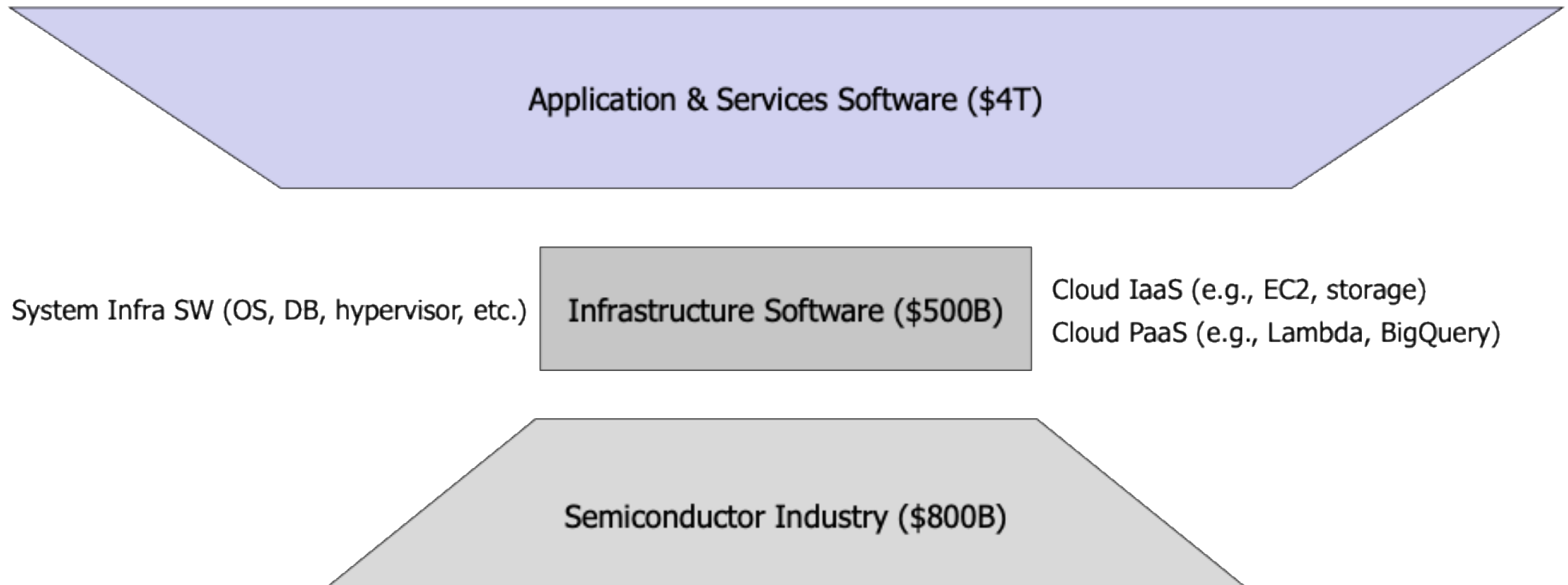
Interconnect Beyond Server

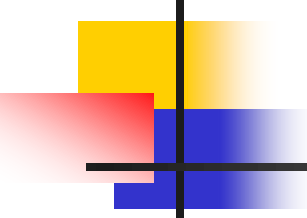




Speed Up Computing

- ❑ So far, we have been focusing on the hardware aspects for speeding up computing

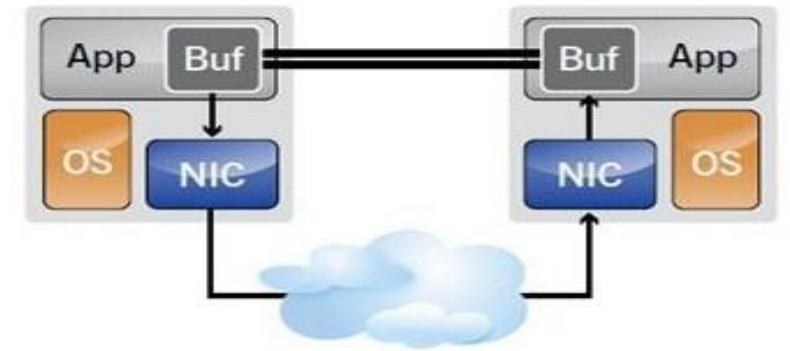
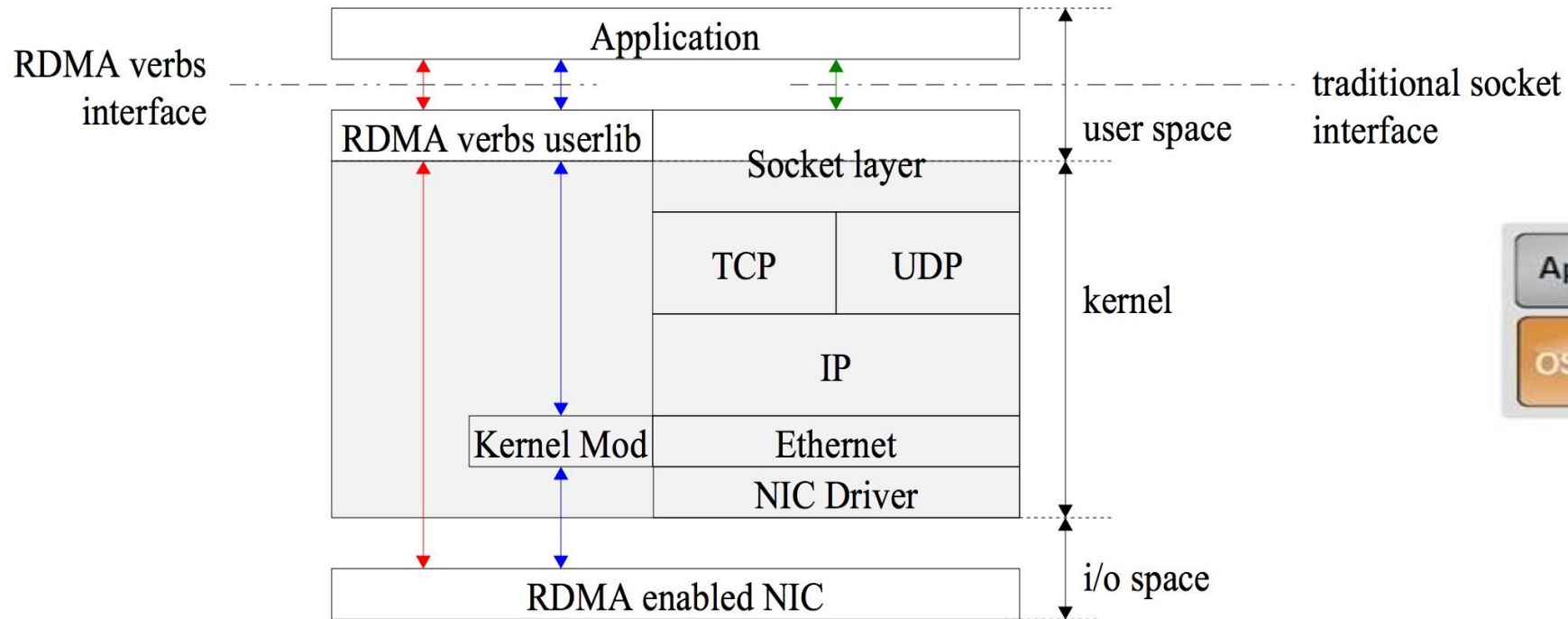




Speed Up Computing: Infra SW

- ❑ **OS, I/O, & Networking:** scheduler, virtual memory & NUMA policy, filesystems & block layer, async I/O, TCP/RDMA stacks, NIC queue/IRQ affinity
- ❑ **Runtimes & Memory Management:** thread pools/OpenMP, work stealing, pinning/affinity, allocators (jemalloc/mimalloc), huge pages
- ❑ **Storage & DB Engines:** B+ tree, LSM tree, cuckoo/Robin-Hood hash, logging, transactions, compaction, caching
- ❑ **Data Layouts & Access Paths:** columnar formats, zone maps & page filter, hot/cold separation and tiering, late materialization

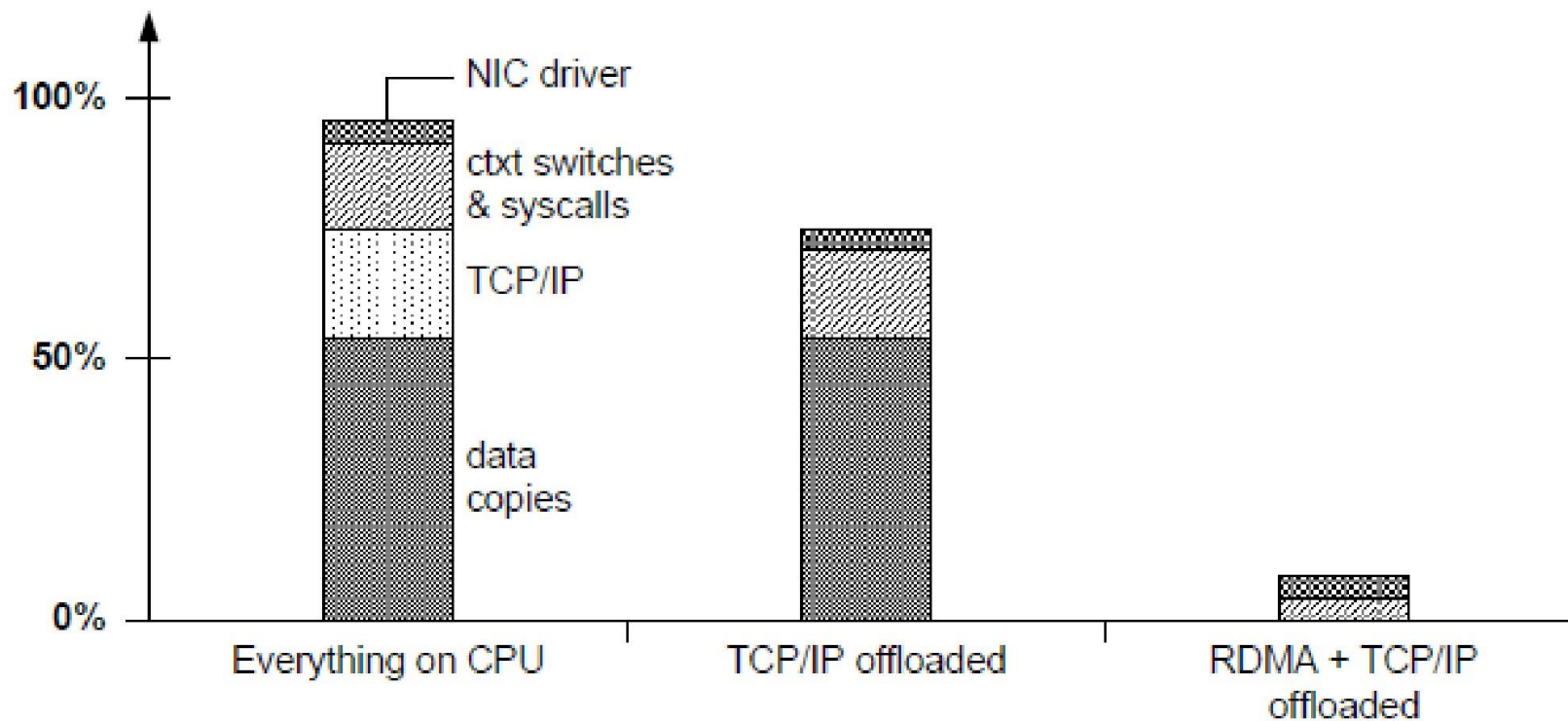
RDMA



- ❑ Involves the kernel only on control path
- ❑ Can access RDMA-enabled NIC directly from user space on data path, bypassing kernel
- ❑ Once data path is established, directly read from/write to buffers

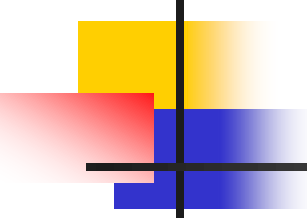
RDMA

□ CPU load



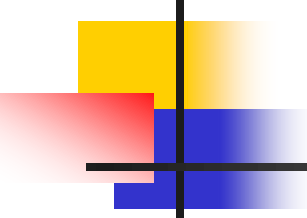
□ RDMA runs at 40Gb/s → 100Gb/s → 400Gb/s → ... 1Tb/s

□ But memcpy run at perhaps <50Gb/s, which may change soon, e.g., Intel data streaming accelerator (DSA)



Infra SW: Data Indexing

- ❑ Data indexing: Data structure for locating and managing/manipulating data (e.g., insert, update, and delete)
- ❑ Computers either process data or manage data, both demand efficient implementation of data indexing
- ❑ Data model: <key, value> pair, where “key” is the ID and “value” is the data
 - Basic operations: GET, PUT, DELETE, SCAN
 - Advanced features: MVCC (multi-versioning concurrency control)



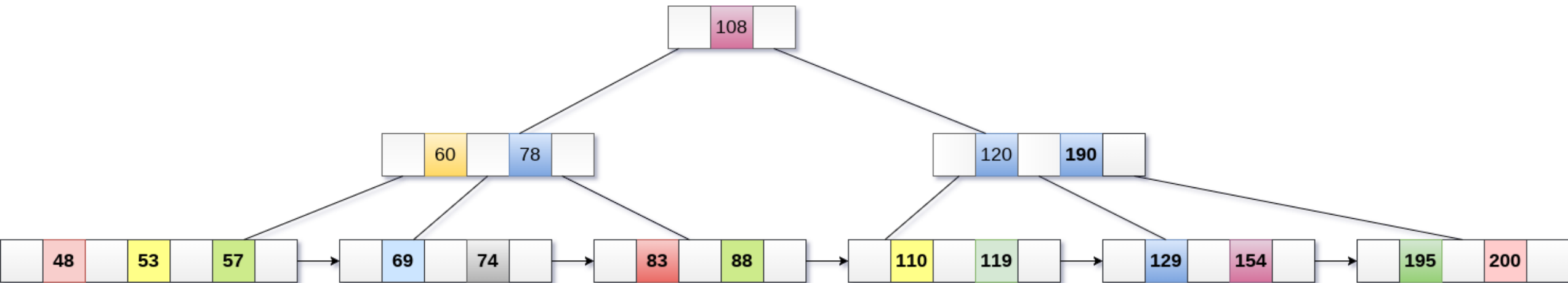
Basic Data Structure

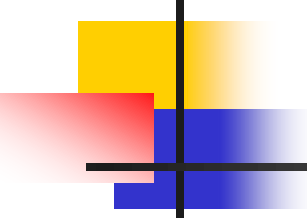
- ❑ A good indexing data structure must be able to efficiently locate data → data must be “sorted” one way or another
- ❑ Natural option: Tree-based structure → directly sort the key
 - Most widely studied and deployed
 - B-tree, log-structured merge (LSM) tree, skip list, red-black tree, R-tree, ...
- ❑ Hash-based structure → sort the hash of the key
 - Good choice if support of scan is not required
 - Widely used by in-memory data store

B-Tree

❑ Developed in 1970's

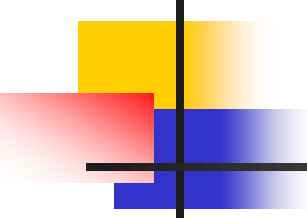
- Self-balancing (height-balanced) tree structure
- Generalizes binary search tree (BST) by allowing nodes with more than two children
- Original B-tree stores <key, value> pairs in all the nodes (branch nodes and leaf nodes)
- Modern implementation (formally called B+ tree) stores <key, value> pairs only in leaf nodes (branch nodes store only keys)





B-Tree

- ❑ B-tree shares many characteristics as BST (e.g., search, insert, delete)
- ❑ Why B-tree since we already have the simple/beautiful BST?
 - Storage devices (SSD, HDD) are block-accessible (512B or 4KB) → match tree node size to storage device block size
 - How about in-memory tree?



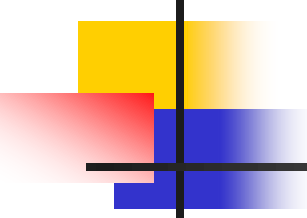
B-Tree

❑ Basic properties

- B-trees are indexes optimized for paged environments without byte accessibility
- B-trees are ordered and permit efficient insertion/deletion
- Some implementations maintain the sort order within each node → binary search
- B-trees are balanced → guarantees uniformly efficient search

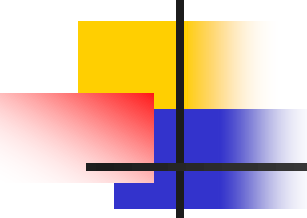
❑ Modern B-tree vs. original B-tree

- By keeping real data in leaf nodes only, modern B-tree enables a much larger node fanout → lower tree height → faster search speed



B-Tree

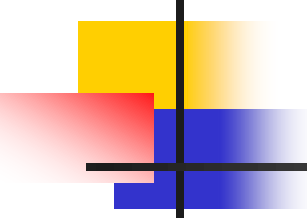
- B-tree with N records, L records per leaf node, and branch node fan-out F
 - B-tree height: $\log_F(N/L)$, logarithmic in the number of records
 - # of comparison operations:
 - ❖ binary search among F child pointers in a branch node: $\log_2(F)$ requires
 - ❖ binary search within one leaf node: $\log_2(L)$
 - ❖ total #: $\log_F(N/L) \cdot \log_2(F) + \log_2(L) = \log_2(N)$
 - Computational complexity primarily depends on the total number of record N



B-Tree

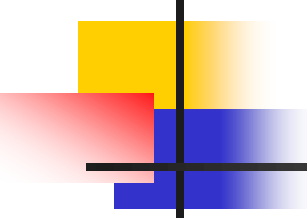
- ❑ In a typical implementation

- B-tree height is usually small, and often more than 99% of all nodes are leaf nodes
- B-tree nodes are filled 50%~100%, permitting insertions and deletions as well as splitting and merging nodes
- Average page utilization after random updates is about 70%



B-Tree

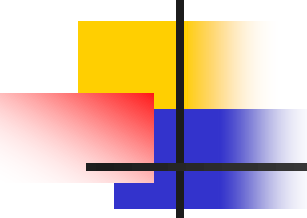
- ❑ Insertion starts from leaf node, and if needed split nodes from leaf nodes towards root → depth of B-tree grows at the root → guarantee perfect tree balance
- ❑ Deletion could trigger node merge
- ❑ B-tree node split and merge are expensive operations (why?)



B-Tree: Data Structures and Algorithms

❑ Node size

- Match the access granularity of underlying memory/storage media
- Match intra-node search latency with node access latency → node size: memory < SSD < HDD



B-Tree: Data Structures and Algorithms

❑ Pre-fix B-trees

- Aims to reduce the memory/storage cost
- Simple idea: analyzes the keys and stores the common prefix only once, truncating it from all keys stored in the node

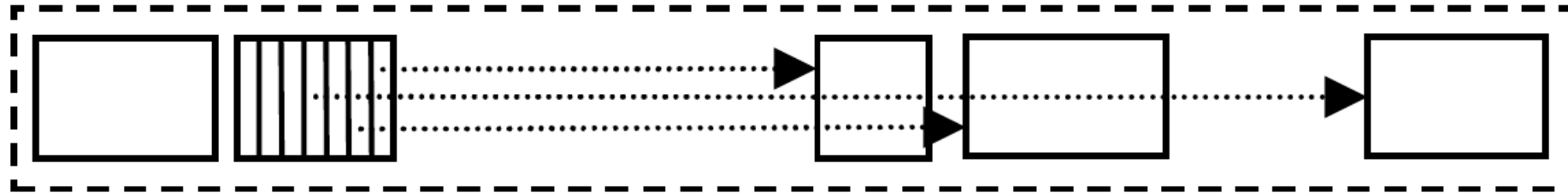
Smith, Jack – 02/29/1924
Smith, Jane – 08/14/1928
Smith, Jason – 06/29/1987
Smith, Jeremy – 03/01/1983
Smith, Jill – 12/31/1956
Smith, John – 10/12/1958
...
Smith, June – 05/05/1903

Prefix = Smith, J
ack – 02/29/1924
ane – 08/14/1928
ason – 06/29/1987
eremy – 03/01/1983
ill – 12/31/1956
ohn – 10/12/1958
...
une – 05/05/1903

B-Tree: Data Structures and Algorithms

❑ Variable-length records

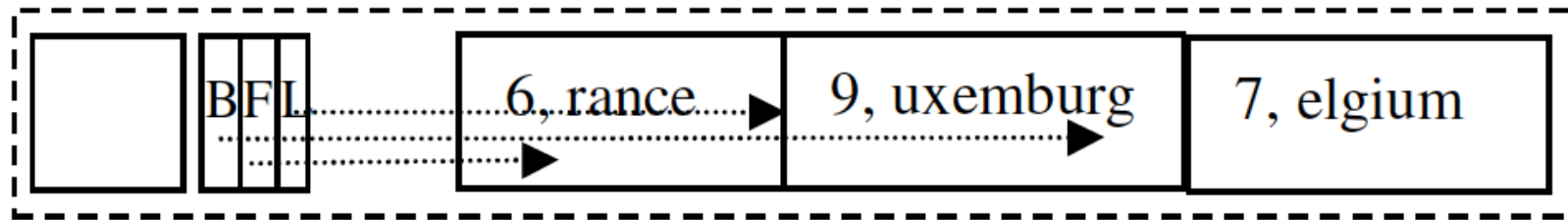
- Use a layer of indirection vector for locating records inside each node
- The entries in the indirection vector are sorted on their search key



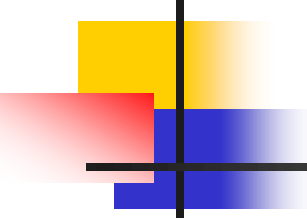
B-Tree: Data Structures and Algorithms

❑ CPU cache

- Cache miss penalty is large
- Keep the indirection vector compact/small → reduce data cache miss rate



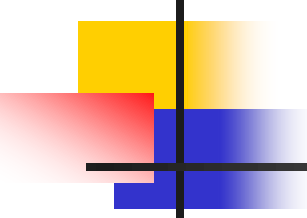
- Organize indirection vector as a B-tree of cache lines, which may cut search time and cache misses by half



B-Tree: Data Structures and Algorithms

❑ Space management

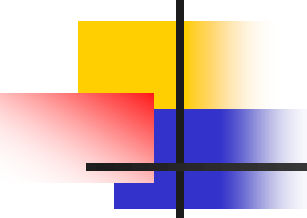
- How should deletion be handled?
- How should insertion be handled?
 - ❖ A new record must be placed where its key belongs, cannot be placed anywhere else
 - ❖ When an insertion fails due to insufficient space in the node, what could we do?
- To avoid or delay node split, one may leave some free space (e.g., 10%) in every node during index creation, bulk loading, and defragmentation



Issues of B-tree

❑ Space efficiency

- Free space within nodes ($\sim 30\%$) due to random insertion/deletion
- Low compression efficiency: compressed nodes must be aligned to storage blocks

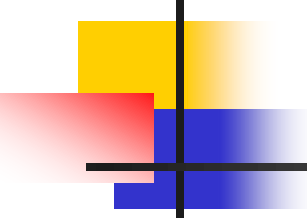


Issues of B-tree

☐ Write amplification

- When a node is modified (insertion/update/delete), the entire node must be written to storage
➔ large write amplification for small data modification
- Large write amplification ➔ lower data modification throughput

☐ Any other issues?

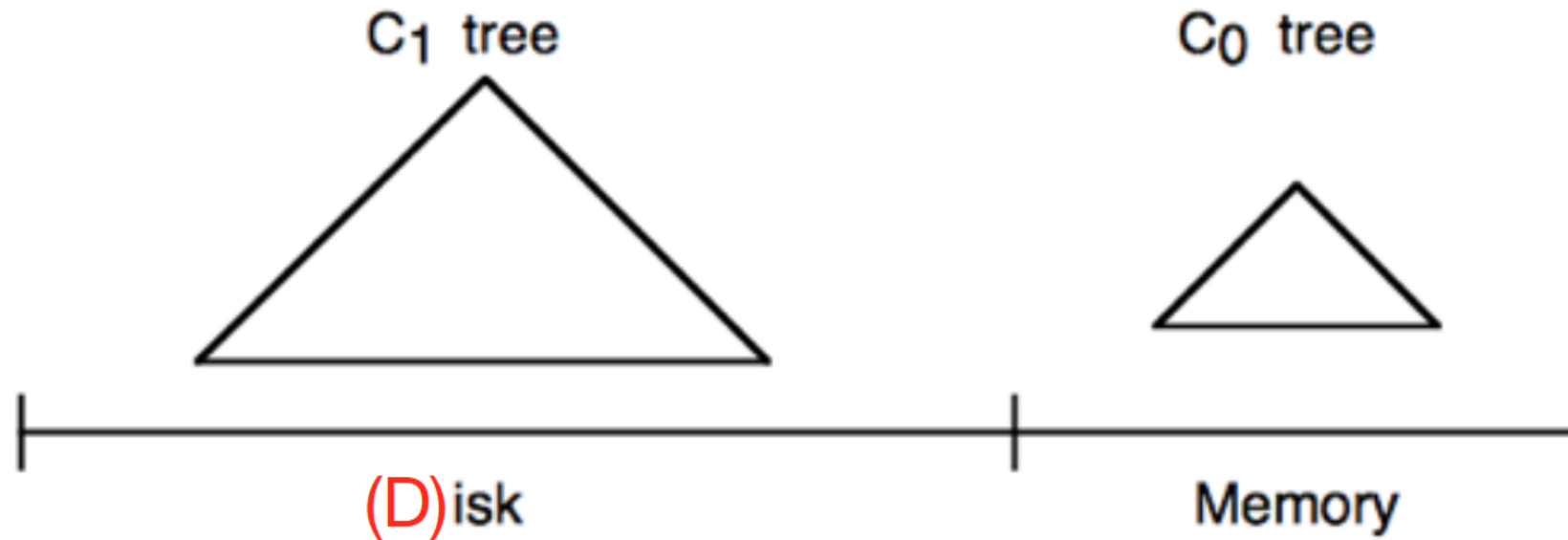


Log-Structured Merge (LSM) Tree

- ❑ Observation: The problems of space efficiency and write amplification fundamentally come from the nature of **in-place update** of B-tree
 - insert/update/delete records within nodes
 - re-write one node independently from other nodes
- ❑ Intuition: Completely obviate in-place update, but then how we could keep data sorted?
- ❑ LSM-tree: a paper published in 1996 (by O'Neil, Cheng, and Gawlick)
 - First well-known implementation: LevelDB by Google
 - Being widely used outside relational databases: RocksDB, HBase, Cassandra, ...
 - (Unsettled?) debate on B-tree vs. LSM-tree

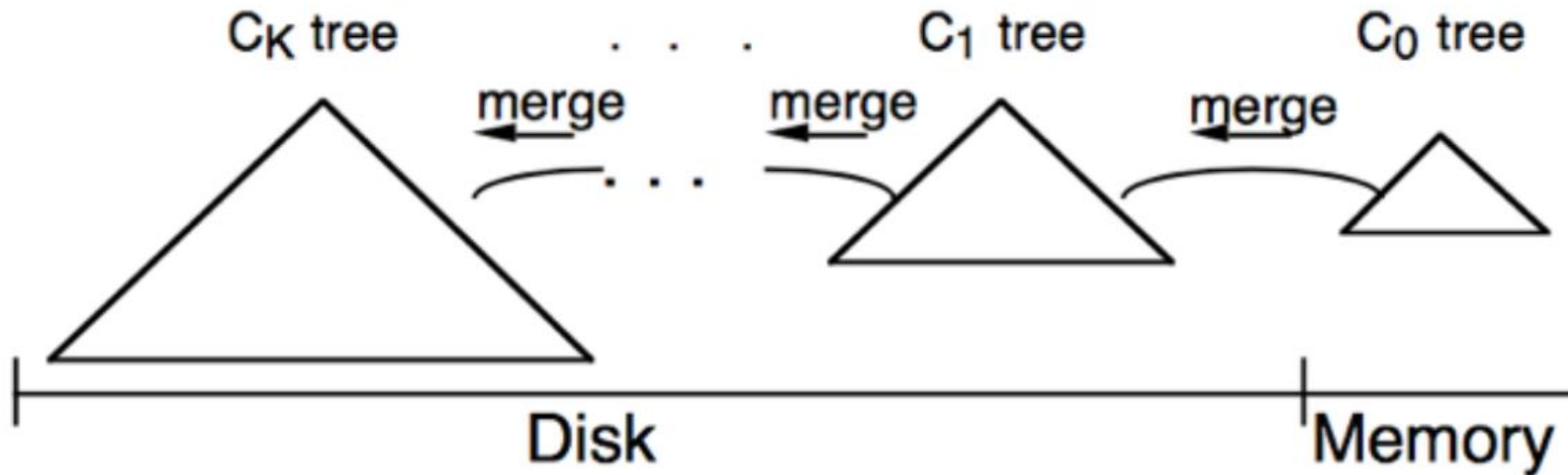
Log-Structured Merge (LSM) Tree

- ❑ An LSM-tree comprises a hierarchy of trees of increasing size
 - All new data are first inserted into in-memory tree (C_0)
 - Larger on-disk trees (C_i) hold data that do not fit into memory



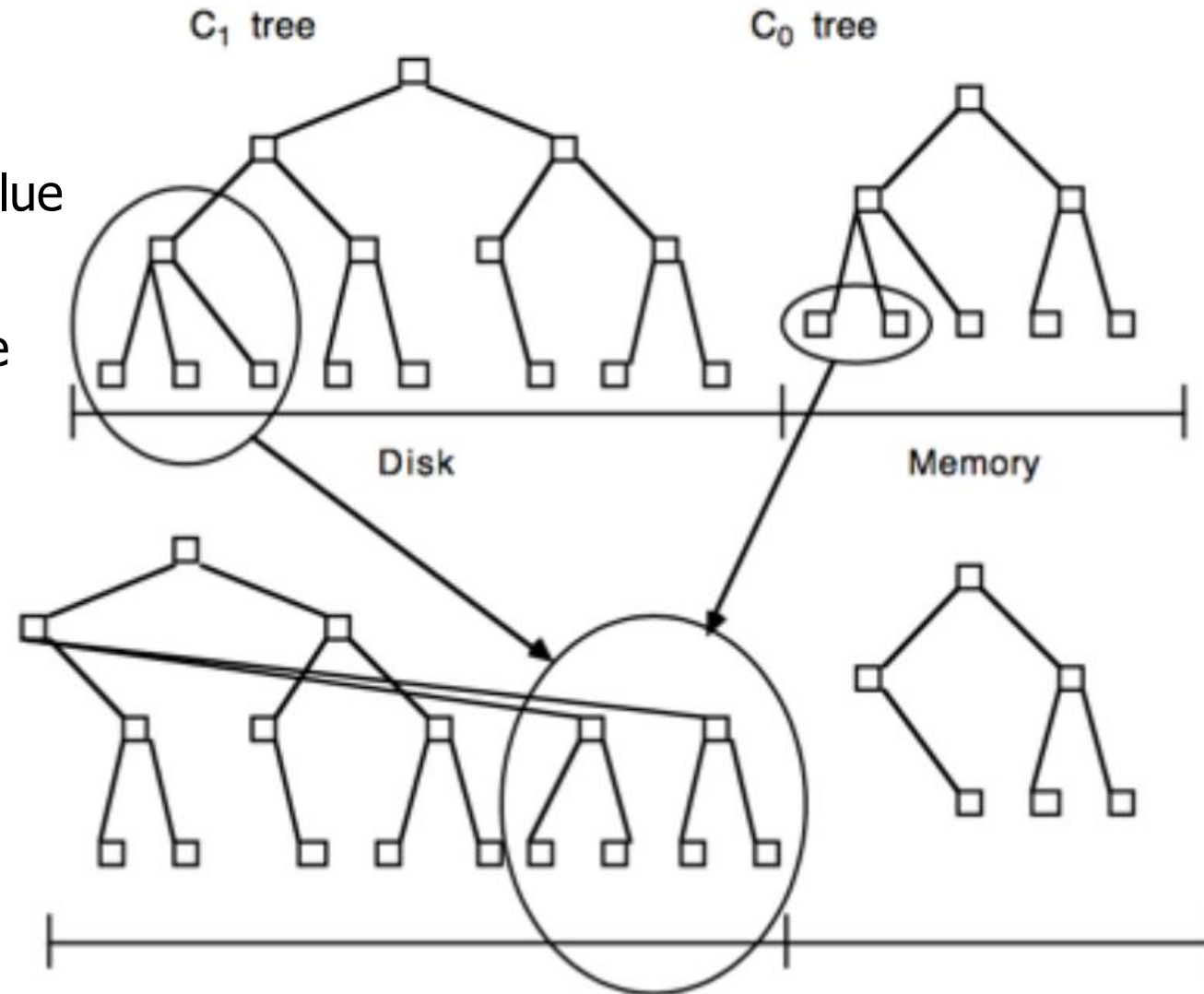
Log-Structured Merge (LSM) Tree

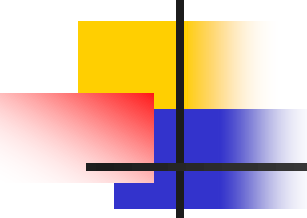
- When a tree exceeds its size limit, its data are merged and rewritten
 - Higher level is always merged into next lower level (C_i merged with C_{i+1})



Log-Structured Merge (LSM) Tree

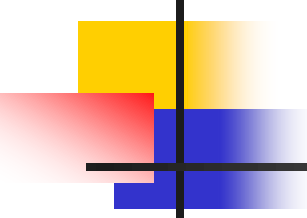
- ❑ We can efficiently merge two sorted structures (remember MergeSort?)
- ❑ When merging two levels, newer version key-value pair replaces older one
 - The newest version of a key-value pair is the version nearest to the top of LSM-tree
- ❑ Key-value pairs in each C_i are nicely sorted





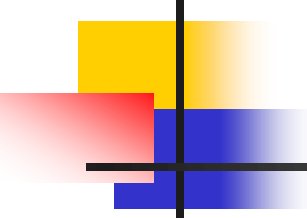
LSM-Tree Basic Operations

- ❑ Insert: insert the key-value (KV) pair into the in-memory level C_0
 - Don't care about lower levels, as long as the newest version is closest to the top
 - If an old version of KV pair exists in the top level, we must replace it
 - If C_0 exceeds its size limit, compact (merge)



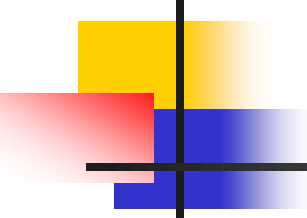
LSM-Tree Basic Operations

- ❑ Delete: insert a **tombstone** into the in-memory level C_0
 - A tombstone is a “logical delete” of all KV pairs with the same key ***k***
 - When we merge a tombstone with a KV pair, we simply delete the KV pair
 - When we merge a tombstone with a tombstone, just keep one
 - When can we delete a tombstone?
 - ❖ At the lowest level
 - ❖ When merging a newer KV pair



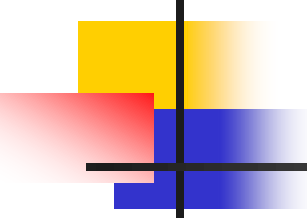
LSM-Tree Basic Operations

- ❑ Point search: Start from the in-memory level C_0 , and continue towards the lowest level until
 - find a KV pair with the key ***k***,
 - find a tombstone with key ***k***, or
 - reach the lowest level and fail to find any match
 - Worse case: traverse every level in the LSM-tree



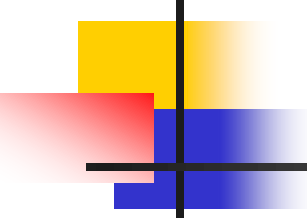
LSM-Tree Basic Operations

- ❑ Range scan: We must search every level $C_0, C_1, \dots$
 - merge the returned KV pairs from all the levels together
 - newest version wins
- ❑ Trade read amplification for write amplification



LSM-Tree: LevelDB

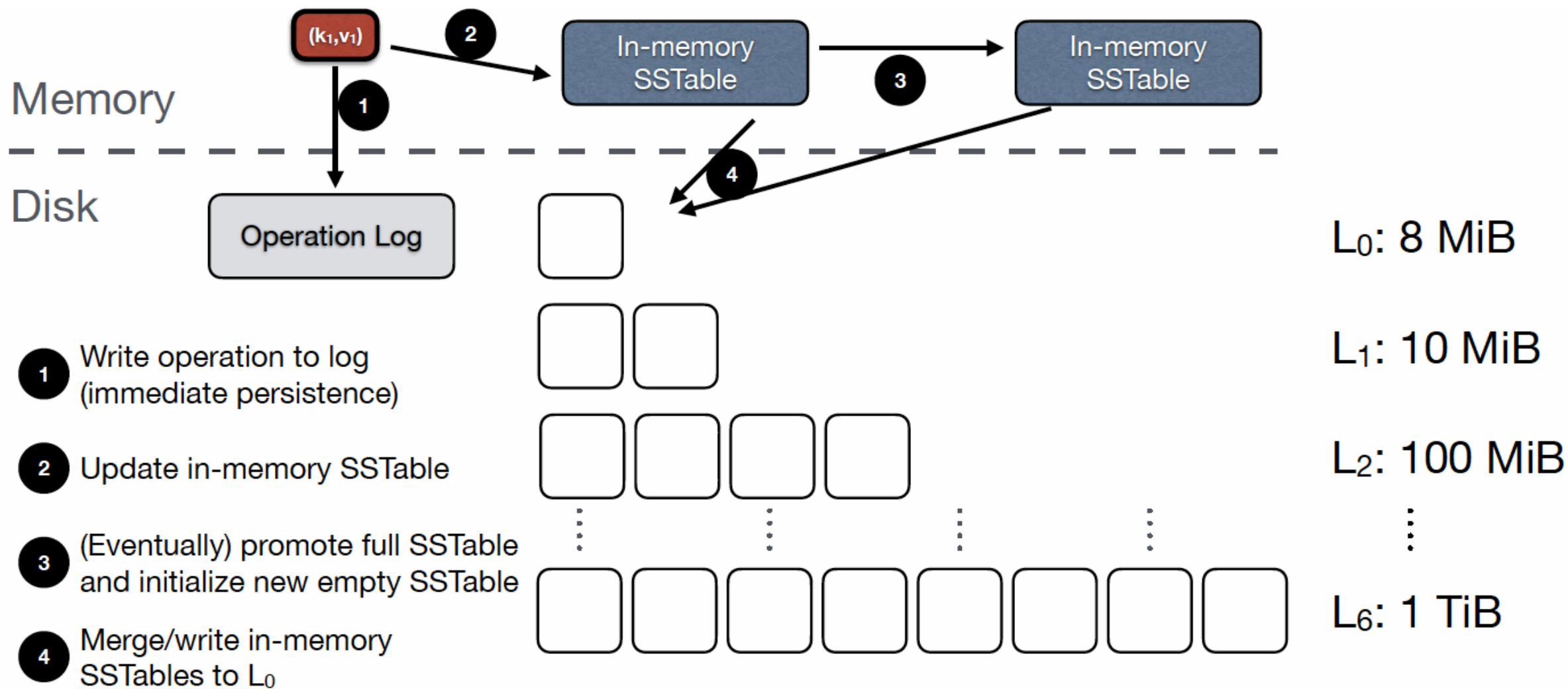
- ❑ LevelDB consists of multiple levels of SSTables (Sorted Strings Tables)
 - Each level consists of multiple blocks with non-overlapping key ranges, all the KV pairs within one block are sorted
 - Each block can be compressed to reduce storage space
 - All the SSTables at the same level have non-overlapping key ranges
 - Each SSTable is stored as an **immutable** file (typical size of few MBs)

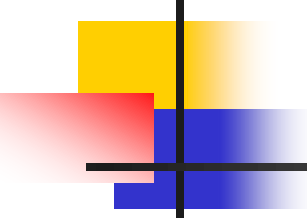


LSM-Tree: LevelDB

- ❑ The **growth factor** describes the scaling of level size
 - Let F be the growth factor (fanout), and M be the size of the first level (e.g., 10MiB) → the i -th level C_i has total size $F^i M$
- ❑ Each SSTable also contains an index block that records the key range of all the data blocks
 - cache all (or most) index blocks in DRAM to accelerate search

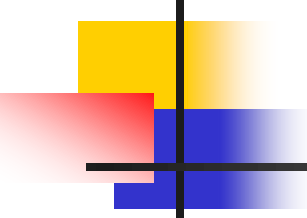
LevelDB Example





LSM-Tree Compaction

- ❑ A topic widely studied in the research community over recent years
 - Different implementations could lead to (largely) different write amplification
- ❑ Background operation to manage the levels in LSM-tree, ideally we would like
 - Write all data sequentially for fast inserts
 - Keep all data sorted for fast searches
 - Minimize the number of levels we must search per query (low read amplification)
 - Minimize the number of times we write each KV pair (low write amplification)
- ❑ Unfortunately, there are fundamental trade-offs and conflict
 - e.g., see “Designing Access Methods: The RUM Conjecture”, 2016

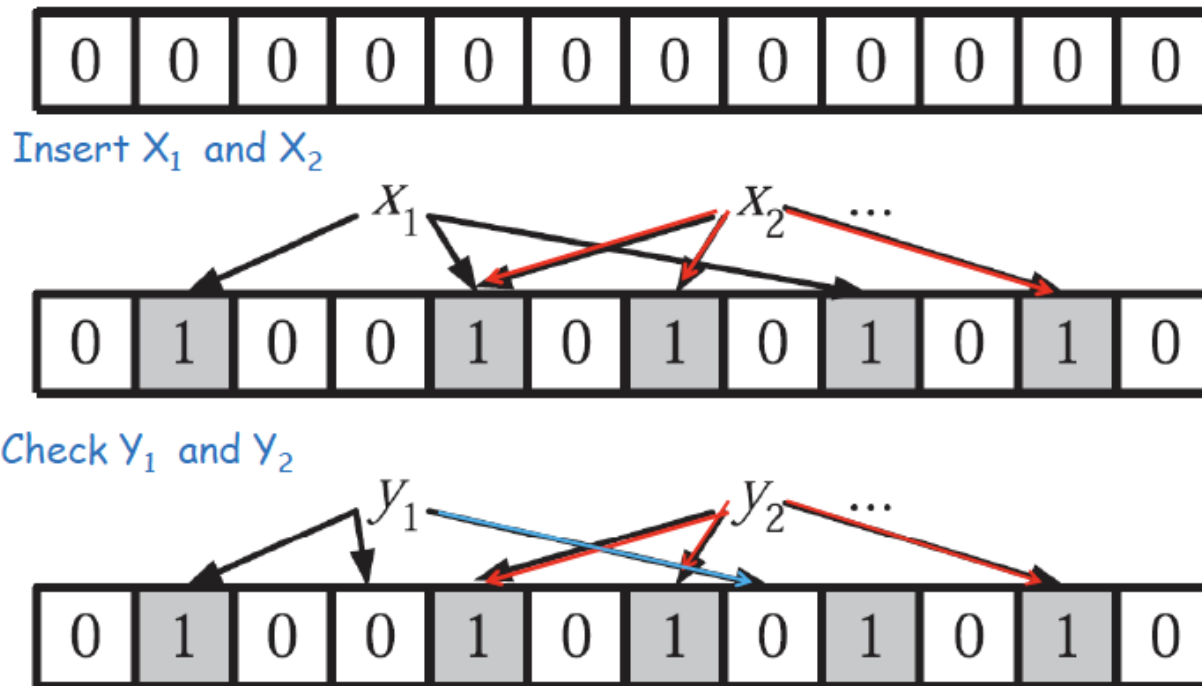


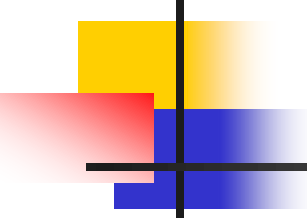
LSM-tree: Further Optimization

- ❑ Add Bloom filters in SSTables to reduce read amplification
 - Bloom filter: a simple probabilistic data structure that can test whether an element is a member of a set
 - When generating each SSTable, create a Bloom filter for the set consisting of all the keys in this SSTable (and store the Bloom filter together with the SSTable)
 - Keep all the Bloom filters in memory → avoid unnecessary search (read IOs)
 - Only helps point query, not range scan

Bloom Filter: Basics

- ❑ A Bloom filter is an array of m bits representing a set $S = \{x_1, x_2, \dots, x_n\}$ of n elements
 - k independent hash functions $h_1, \dots, h_k$ with range $\{1, 2, \dots, m\}$
 - For each element x_j in S , the bit at the position $h_i(x_j)$ in the array is set to 1, for $1 \leq i \leq k$





Bloom Filter: False positive probability

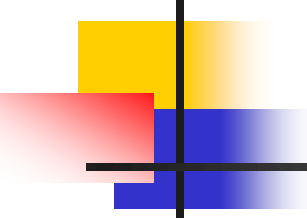
- ❑ After all members of S have been hashed to a Bloom filter, the probability that a specific bit is still 0 is

$$p' = \left(1 - \frac{1}{m}\right)^{kn} \simeq e^{-kn/m} = p$$

- ❑ For a non-member, it may be found to be a member of S (all of its k bits are nonzero) false with positive probability

$$(1 - p')^k \simeq (1 - p)^k$$

- ❑ How to choose the k in order to minimize the false positive probability?

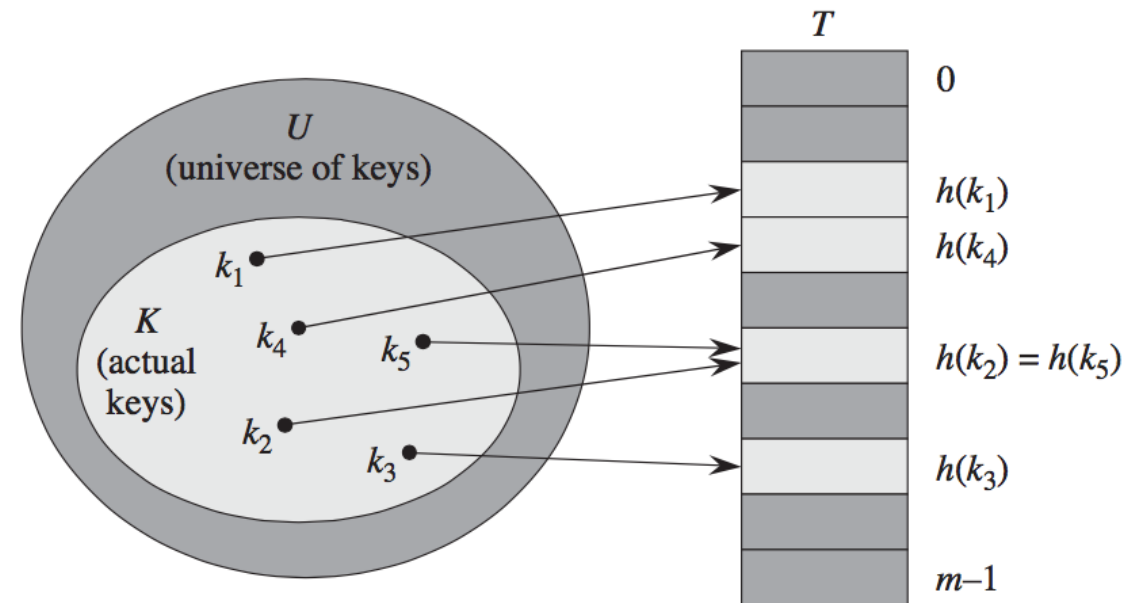
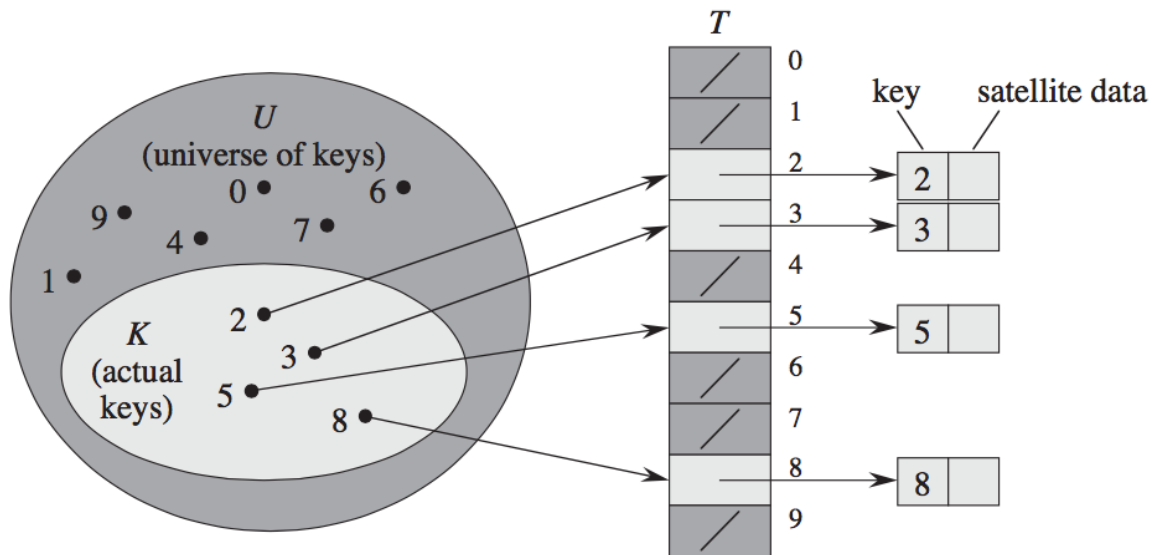


Bloom Filter

- ☐ What are the major advantages of Bloom filter over other data structures (e.g., tree, array, list, hash table) for representing sets?
- ☐ What prices does Bloom filter pay to get these advantages?

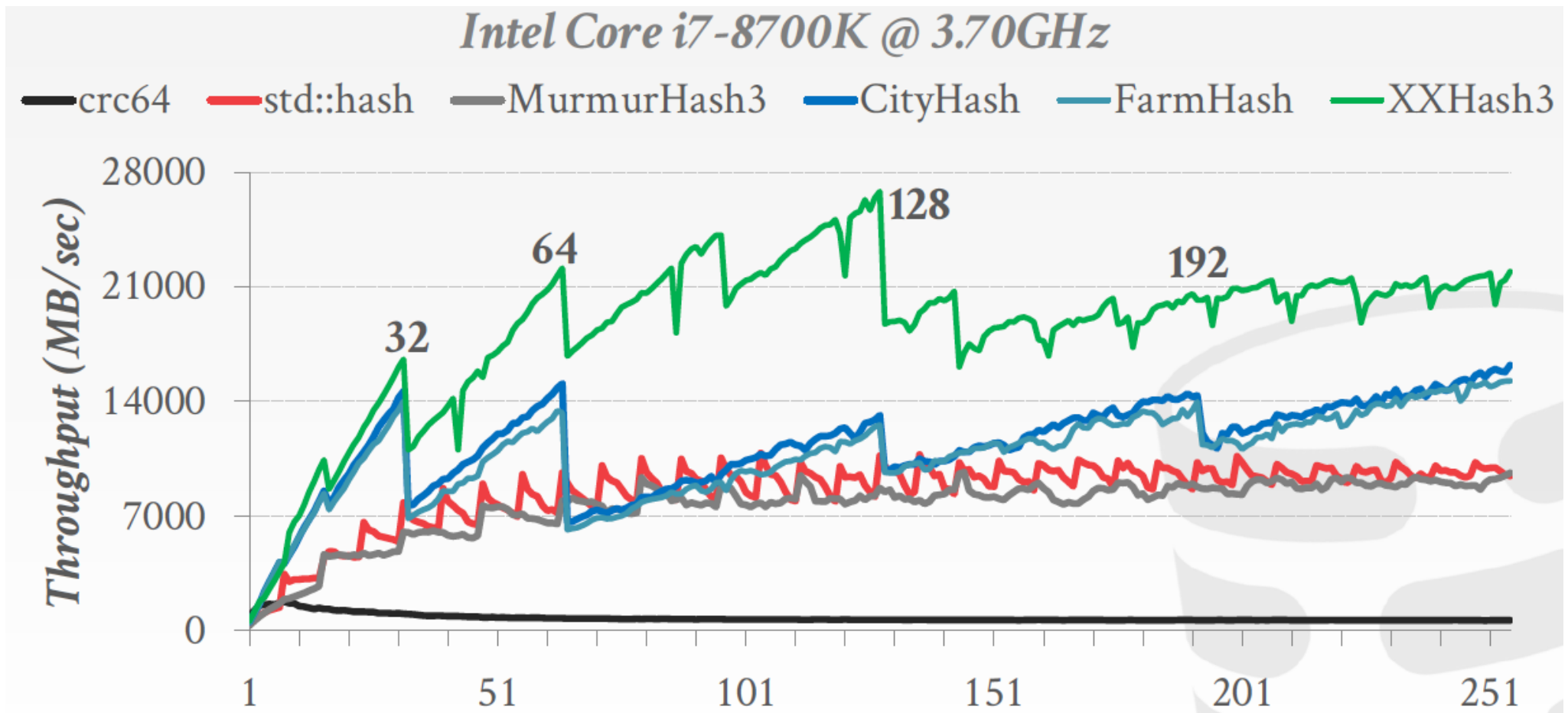
Hash Table

- ❑ A very simple idea to build data index with very short query latency
 - Hash function: map a large key space into a smaller domain, trade-off between hashing speed and collision rate
 - Hashing scheme: determine how to handle key collisions, trade-off between allocating a large hash table vs. additional instructions to find/insert keys.



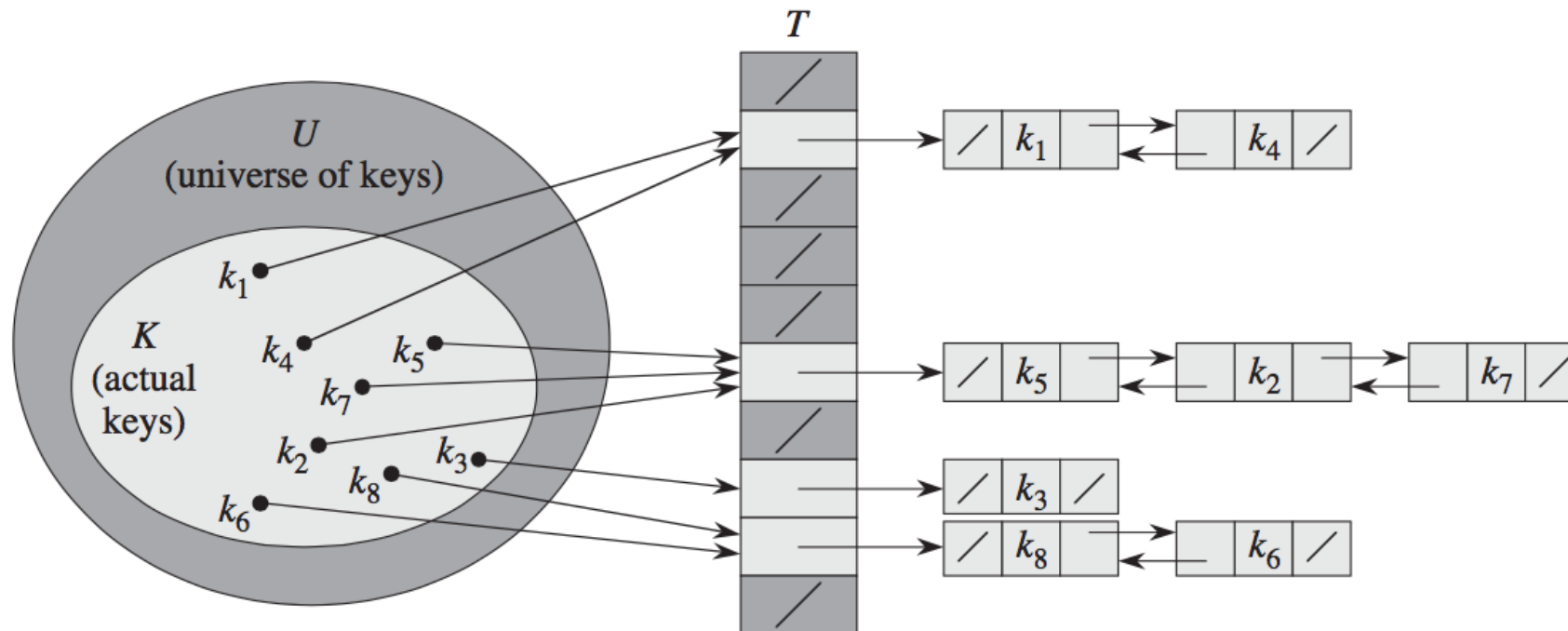
Hash Function

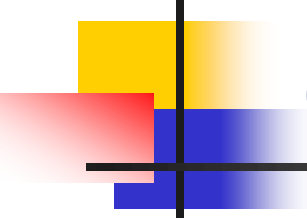
- ❑ Cryptographic hash functions (e.g., MD5, SHA) are not appropriate
- ❑ Popular hash functions: CRC-64, MurmurHash, CityHash, XXHash, FarmHash



Hashing Scheme

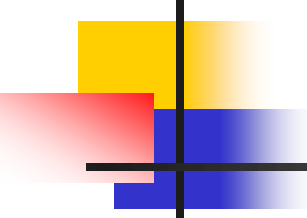
- ❑ Chaining vs. open addressing
- ❑ Load factor: Given a hash table with m slots that stores n elements, its load factor α is n/m





Open Addressing

- ❑ Each entry in the hash table contains at most one element, i.e., load factor $\alpha \leq 1$.
- ❑ Basic idea: We successively examine, or probe, the hash table cells until we find an empty slot to insert
 - Linear/quadratic Probe Hashing
 - Robin Hood Hashing
 - Cuckoo Hashing

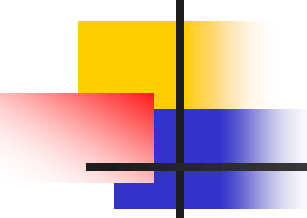


Linear/Quadratic Probe Hashing

- ❑ Given the original hash function $h(k)$, probe the hash table using

$$h(k, i) = (h(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m, \quad i = 0, 1, \dots, m-1$$

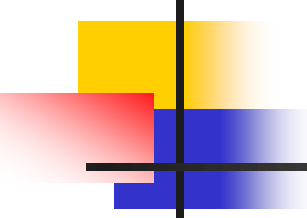
- ❑ Linear probe hashing: $c_2=0$, c_1 is typically 1
- ❑ Quadratic probe hashing: $c_2 \neq 0$
- ❑ Search starting from $i=0$, until find a match or reach an empty entry



Linear/Quadratic Probe Hashing

❑ How to deal with delete?

- Solution 1: add a tombstone at the deleted entry
- Solution 2: move all subsequent KV pairs to override the deleted entry, which may trigger more deleted entries and more movement



Robin Hood Hashing

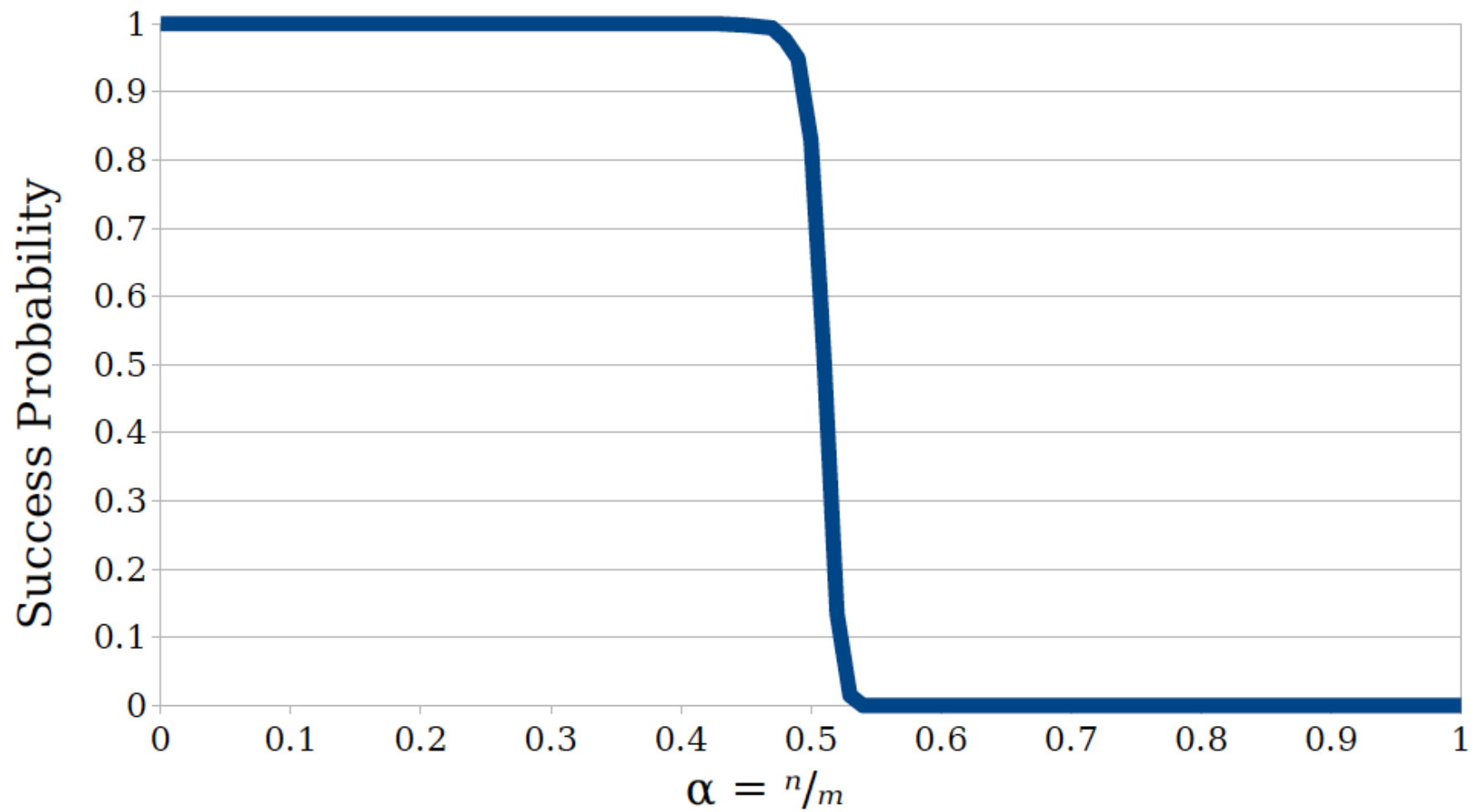
- ❑ A variant of linear probe hashing that steals slots from "rich" keys and give them to "poor" keys
 - Each key tracks the number of positions from its ideal position
 - On insert, a key takes the slot of another key if the first key is farther away from its ideal position than the second key

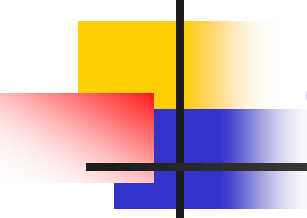
Cuckoo Hashing

- ❑ Use two (or even more) hash tables with different hash functions
 - On insert, check every table and pick anyone that has a free slot
 - If no table has a free slot, evict the element from one of them and then re-hash it to a new location
 - An insertion fails if the displacements form an infinite cycle
 - Rehash with different hash functions
- ❑ Longer average search latency than the others (2 cache misses vs. 1 cache miss), but it guarantees the worst-case search latency



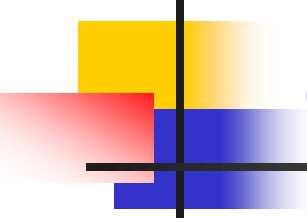
Cuckoo Hashing





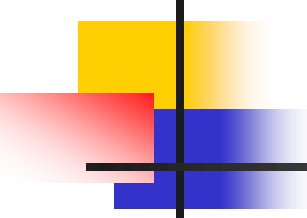
Cuckoo Hashing

- ❑ Insertions take expected time $O(1)$, under the assumption that $n = \alpha \cdot m$ for $\alpha < 1/2$
 - Proof is quite complicated and involves graph theory
 - Represent Cuckoo hash table as a graph
 - ❖ Each table slot is a node
 - ❖ An edge connects two nodes that one element can be placed into
 - Worst-case insertion time depends on the largest connected component (CC)
 - ❖ Given a random (multi)graph G with V nodes and E edges, if $E < \alpha \cdot V$ for $\alpha < 1/2$, then the expected size of a CC in a randomly-built graph is $O(1)$



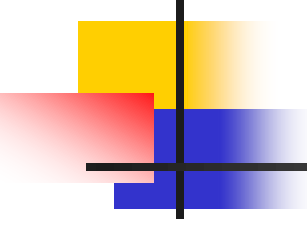
Cuckoo Hashing

- ❑ A cuckoo hash table with n elements requires a table of size n/α , with $\alpha < 1/2 \rightarrow$ at least 50% of the table slots will be empty, a big waste!
- ❑ It is a fundamental property of random graphs; exceeding this threshold makes failure ***almost*** certain
- ❑ How can we push past this to improve cuckoo hashing space usage?



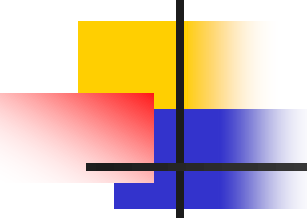
Hash Table

- ❑ Load factor: Trade-off between memory usage and speed performance
- ❑ Rehash: Re-build the hash table to increase or decrease the table size to keep the load factor in a reasonable range
 - How to reduce its impact?



Hash Table

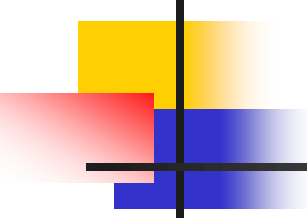
- ❑ To better utilize CPU cache: group multiple elements in one bucket for chained hashing
- ❑ Comparison with B-tree:
 - ✓ Shorter search/insert/delete latency → higher speed
 - x Cannot support scan
 - x Memory fragmentation in the presence of variable-length data
 - x Difficult to support larger-than-memory dataset



Hash Table

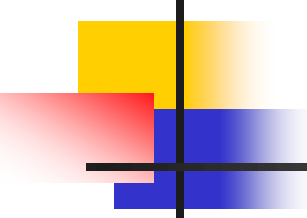
❑ How to handle memory fragmentation?

- Apply slab memory allocation (partition memory into segments with different slab size)
- Store the real KV pairs in the most appropriate slabs, and use the pointer to each KV pair as the key to build an in-memory hash table
- Notable example: Memcached



Hash Table

- ❑ How to support larger-than-memory dataset?
 - Store the real KV pairs on SSD, and use the address of on-SSD KV pairs as the value to build an in-memory hash table
 - Now the question becomes: How to manage the on-SSD dataset?



Remarks

- ❑ Most important indexing data structure: B-tree and hash table
- ❑ Very heavily studied and widely deployed in the real world
 - Thousands of research papers and hundreds of PhD thesis
 - Many variants have been developed
 - Cornerstone of the entire SW infrastructure
- ❑ Which one should you choose?
 - You must fully understand your target application workload characteristics
- ❑ Multi-core & heterogeneous computing + New memory and storage technologies
 - Innovation opportunities